

MFDStudio - Detailed User Guide

design in `mfd\_editor`, C++17 API generation, live client integration, framebuffer plugin, and launch scripts

Document generated on 2026-05-07 20:59

Scope: editor, architecture, generator, client API, framebuffer plugin, launch scripts

Target audience: MFD authors, C++ integrators, and runtime teams

Target document:

- MFD page and window authors
- C++ integrators who must control a window live
- teams who want to capture the framebuffer via a DLL plugin

Table of Contents

Table of Contents	2
1. What is MFDStudio	7
1.1 The main components	8
1.2 The minimum mental model	9
1.3 Who does MFDStudio serve?	11
1.4 What MFDStudio is not	11
2. Detailed Architecture	12
2.1 Depot modules	12
2.2 Author / runtime / client separation	12
2.3 The role of the file .generated.map	13
2.4 Runtime loop and feedback	13
2.5 Author model	14
2.6 Practical consequences for integration	14
3. Prerequisites and tools to build	15
3.1 Recommended Toolchain	15
3.2 Build local recommends	15
3.3 Useful binaries as needed	15
3.4 Tree structure to know	15
4. Use mfd_editor to design a window and its pages	16
4.1 Launch the editor	17
4.2 Anatomy of the interface	17
4.3 Create a new window	17
4.4 Configure UDP transports	17
Incoming commands to window	18
Feedback coming out of runtime to clients	18
4.5 Create a first page	18
4.6 Save what the editor writes	18
4.7 Add, choose and organize pages	19
4.8 Design of a page: layers, static reticles, dynamic bindings, strobe	19

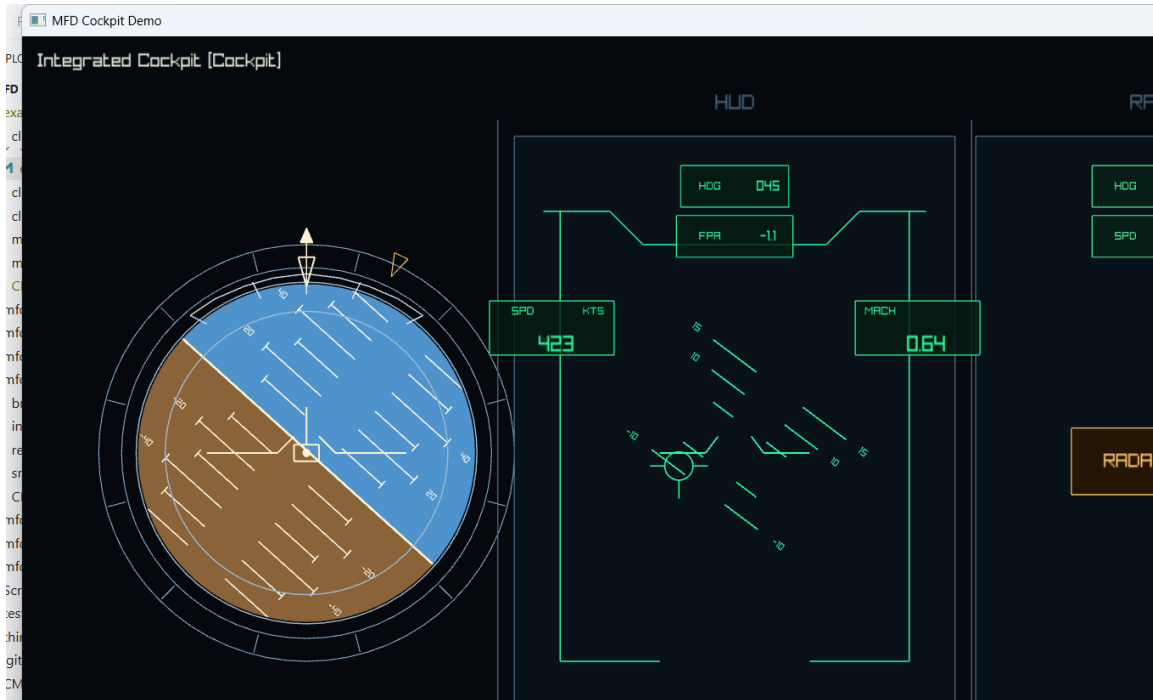
Page layers	19
Static reticles	19
Dynamic reticle bindings	19
Page strobe.....	19
4.9 Important page fields.....	19
4.10 Supported primitives for constructing reticles.....	20
4.11 Selection and direct editing on the canvas	21
4.12 Menu View from the preview	21
Fullscreen preview	21
4.13 Layer Focus Mode	21
4.14 Import an existing page	22
4.15 Delete or detach a page	22
4.16 Properly rename a shared page	24
4.17 Properly renaming a shared reticle template	25
4.18 Highlight use of a template.....	25
4.19 Export of design documentation.....	25
4.20 Recommended authoring checklist	26
5. Generate the client API from the generator.....	27
5.1 Entry, exit, contract	27
5.2 Official CMake macro	27
5.3 client_api_generate_ui arguments.....	27
5.4 What the CMake macro does	28
5.5 Underlying Python script.....	29
5.6 Important generation rules.....	29
5.7 Rules for exposing primitives.....	29
5.8 What the generated API contains	30
5.9 What it contains .generated.map	30
5.10 Example of integration into a CMake target	31
5.11 When to regenerate	31
5.12 Common causes of generation failure	31
6. Use the generated API to communicate with a window	32
6.1 Two layers to distinguish.....	32
6.2 Load window configuration	33
6.3 Create the CommandClient.....	33
6.4 Generated UI root.....	33

6.5 Page wrappers	34
6.6 Activate a page and adjust its view	34
6.7 Modify the overall window display	35
6.8 Mutate a static reticle	35
Common reticle surface	35
6.9 Using exposed primitive handles	36
Common primitive surface	36
Specialized surfaces	36
Example of exposed primitive.....	37
6.10 Manage dynamic reticles	37
Surface of one generated dynamic set.....	37
What the dynamic set hides	37
6.11 Publish in batches	38
Batch without sequence	38
Batch with sequence and mappingHash.....	38
6.12 Use LatestBatchPublisher.....	38
6.13 Use strobe and feedback	38
6.14 What feedback guarantees	39
6.15 Client sanitization and hygiene.....	39
6.16 When to use raw API	39
6.17 Minimal end-to-end example	39
7. Write a framebuffer capture DLL plugin.....	41
7.1 Why a stable C ABI	41
7.2 Mandatory entry point.....	41
7.3 Structure MfdWindowFramebufferPluginApi.....	41
7.4 Structure MfdWindowFramebufferPluginInfo	42
7.5 Structure MfdWindowFramebufferFrame	42
7.6 Validation helpers provided	42
7.7 Plugin lifecycle	43
7.8 Minimal plugin skeleton	43
7.9 Example of repository reference plugin.....	45
7.10 What to do in submit_frame	45
7.11 What not to do.....	45
7.12 Practical consumption strategy.....	45
7.13 CMake build of the plugin.....	46

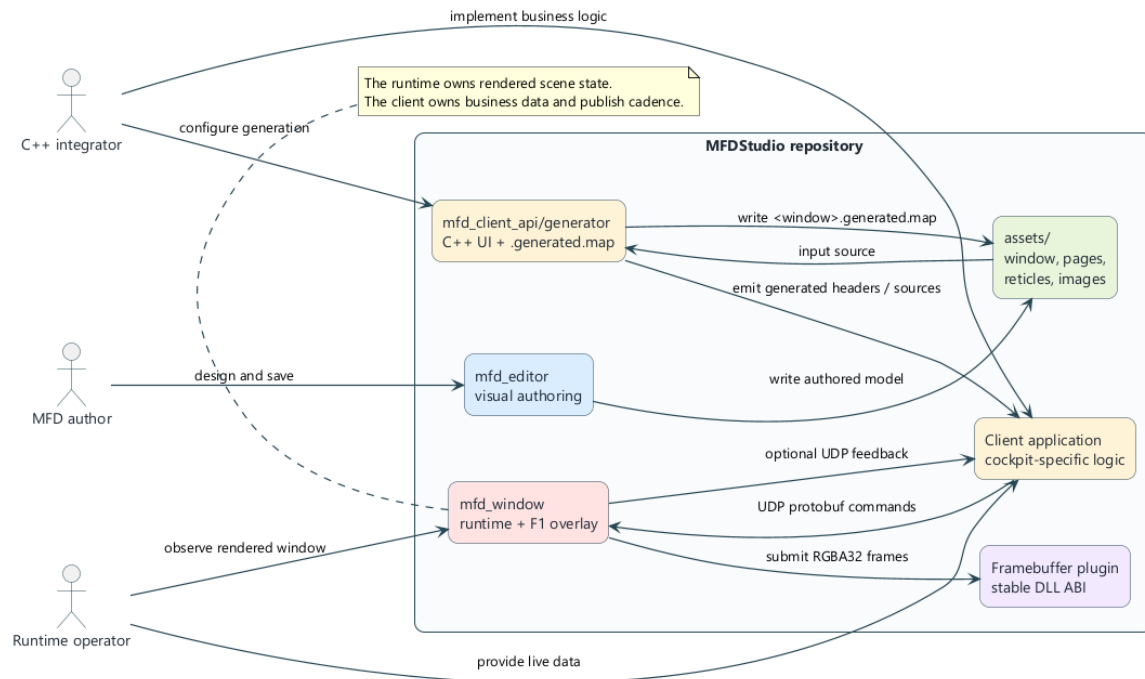
7.14 Recommended testing strategy	46
8. Launch a window with a script and a framebuffer plugin	47
8.1 Use of the script	49
8.2 Supported arguments	49
8.3 Runtime resolution	49
8.4 Plugin resolution	49
8.5 Example with the minimal script from the repository	50
8.6 Explicit example with custom plugin.....	50
8.8 Write your own project launch script	50
8.9 Launch checklist with plugin	50
8.10 If the plugin does not load	51
9. Overall Checklist Recommends	52
9.1 Complete project workflow	52
9.2 Design errors to avoid	52
9.3 Most important files to know	52
9.4 Conclusion	52

The table of contents will be updated by Word.

1. What is MFDStudio



Cockpit runtime screenshot



MFDStudio Overview

MFDStudio is a C++17 / CMake toolbox for designing, executing and controlling MFD type 2D windows from JSON files. The repository voluntarily separates three responsibilities:

- the content author defines versioned assets: window, pages, reticles, images, fonts

- the runtime `mfd_window` loads these assets and renders the active page
- the external client sends live commands via UDP to modify the runtime state without rewriting the assets

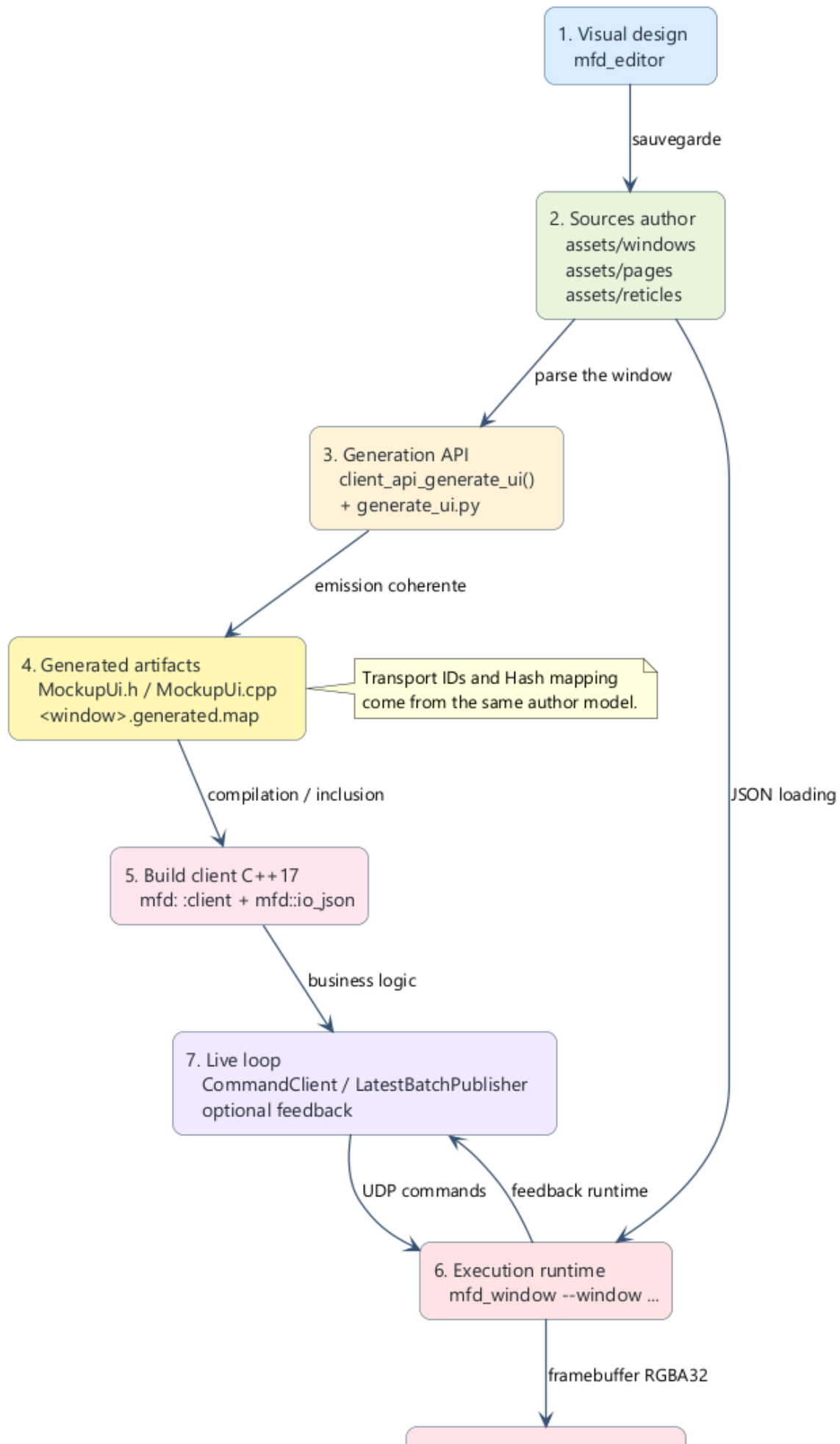
The historical technical prefix of the repository remains `mfd` in namespaces, CMake targets, folders, and public APIs. You must therefore distinguish:

- the product name: MFDStudio
- the technical prefix: `mfd`

1.1 The main components

Component	Role
<code>mfd_editor</code>	visual tool for authoring windows, pages, and reticles
<code>assets/</code>	versioned source of truth
<code>mfd_client_api/generator</code>	generates typed C++ wrappers and the companion <code>.generated.map</code> file
<code>mfd_window</code>	runtime host that opens a JSON window, renders the active page, and accepts live commands
<code>client_mockup</code>	reference client that shows how to use the public API
<code>mfd_window_plugin_api</code>	stable C ABI for framebuffer capture DLL plugins

1.2 The minimum mental model



Pipeline authoring to runtime

The normal flow of an MFDStudio project is as follows:

1. 1. we author a window and its pages in assets/ with mfd_editor
2. 2. we generate a C++ client API specific to this window
3. 3. we compile a business client which uses this generated API
4. 4. we launch mfd_window on the JSON window
5. 5. the client sends UDP commands to the runtime
6. 6. the runtime can send UDP feedback to the client
7. 7. optional, a DLL plugin receives the RGBA32 framebuffer at each frame

1.3 Who does MFDStudio serve?

MFDStudio is suitable when you need:

- cleanly separate graphic authoring and real-time logic
- keep a readable and versionable author model
- drive pages by an external client without embedding the rendering engine in the client
- generate C++ type wrappers from an author window
- plug a framebuffer capture pipeline into a stable DLL ABI

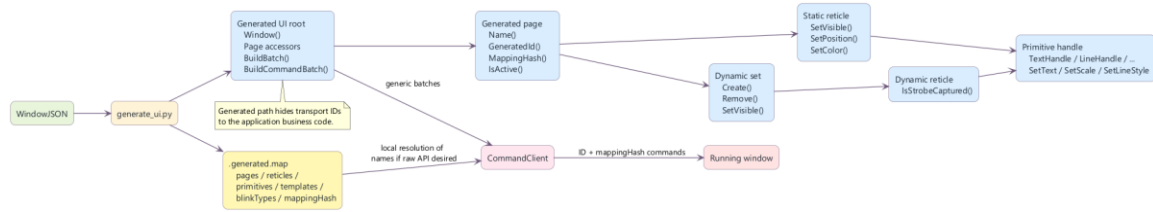
1.4 What MFDStudio is not

MFDStudio is not:

- a general Qt-type UI framework
- a 3D scene authoring system
- a generic network protocol independent of assets
- a plugin system open to any C++ ABI

Important: The runtime remains the owner of the scene state and the rendering. The client remains the owner of the business data and the transmission rate. It is this separation that gives stability to architecture.

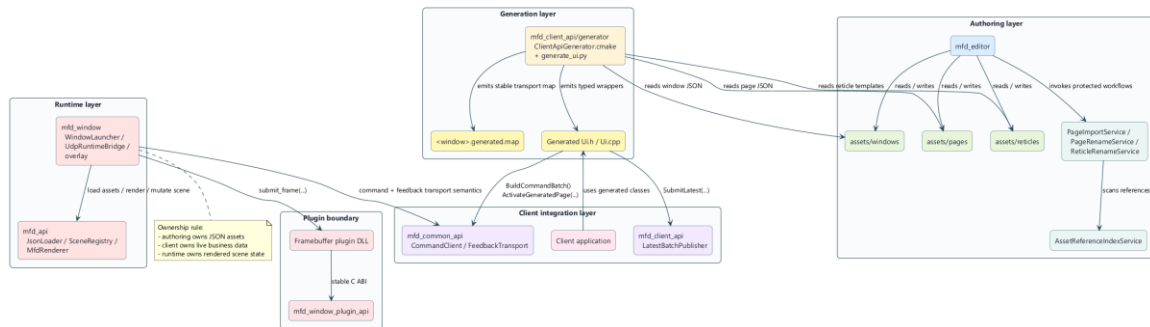
2. Detailed Architecture



API and transport model

The repository architecture is modular. Each module keeps a deliberately narrow responsibility.

2.1 Depot modules



Modular layers of the repository

Module	Primary Responsibility
mfd_common_api	common types, low-level authoring model, transport and commands
mfd_api	loading JSON, runtime, rendering, scene registry
mfd_client_api	client overlay: handles, batching, publication, feedback
mfd_client_api/generator	derives a C++ UI and a transport map from a JSON window
mfd_editor	visual authoring and protected workflows on assets
mfd_window	generic runtime launcher and debug overlay
mfd_window_plugin_api	Stable C contract for framebuffer capture plugins
examples/	reference clients and plugins

2.2 Author / runtime / client separation

The architecture is based on three layers which must not be mixed:

Layer	Possessed	Must not own
-------	-----------	--------------

Authoring	visual structure and JSON assets	live business logic
Runtime	rendering and state authoritarian scene	knowledge of the customer business domain
Customer	live data, cadence, control intentions	the window rendering engine

2.3 The role of the file .generated.map

The generator emits two coherent artifacts:

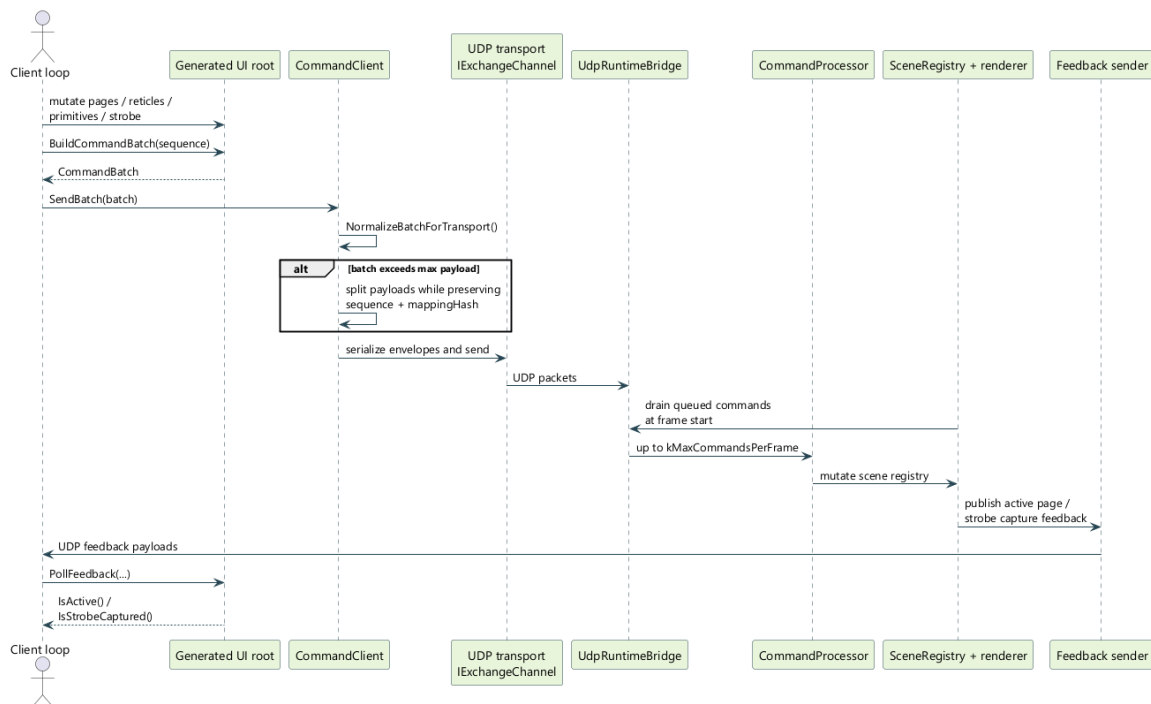
- a C++ header/source generated for the client
- a <window>.generated.map file

The .generated.map contains stable transport IDs for:

- the pages
- static reticles
- the primitives exposed
- dynamic templates
- the blink types

It also contains a mappingHash used to prove that the client and the window are talking about the same author model.

2.4 Runtime loop and feedback



Command sequence and feedback

In the recommended mode:

- a UDP worker receives command packets
- the render thread drains commands at the start of the frame

- the runtime applies the mutations via the CommandProcessor and the scene
- the runtime can send authoritative feedback to the client

This feedback is used in particular to:

- the active page rendered
- the resolved state of the strobe
- capturing a dynamic reticle by the strobe

2.5 Author model

The minimal author model is:

- a window
- a list of pages
- of the staticReticles instances on each page
- a folder of reusable reticle templates
- of the dynamicReticleBindings on the pages
- A strobe optional per page
- of the blinkTypes defined at page level

The logical author and client coordinates are normalized in $[-1, 1]$.

2.6 Practical consequences for integration

For a clean integration:

- authoring the visual structure into JSON
- only expose the necessary primitives to the client
- use the generated API rather than text names everywhere
- preserve CommandClient as final sending layer
- reserve framebuffer plugins for image output, not for business control

3. Prerequisites and tools to build

3.1 Recommended Toolchain

The target deposit in practice:

- Visual Studio 2022
- CMake 3.25 or higher
- C++17
- Python 3 for the generator

3.2 Build local recommends

The preferred local flow is Debug Win32:

```
cmake --preset vs2022-win32
cmake --build --preset debug-win32
ctest --preset test-debug-win32
```

3.3 Useful binaries as needed

To author and test the main flow:

```
cmake --preset vs2022-win32
cmake --build --preset debug-win32 --target mfd_window mfd_framebuffer_stdout_plugin client_mockup
mfd_editor
```

For one specific generated client:

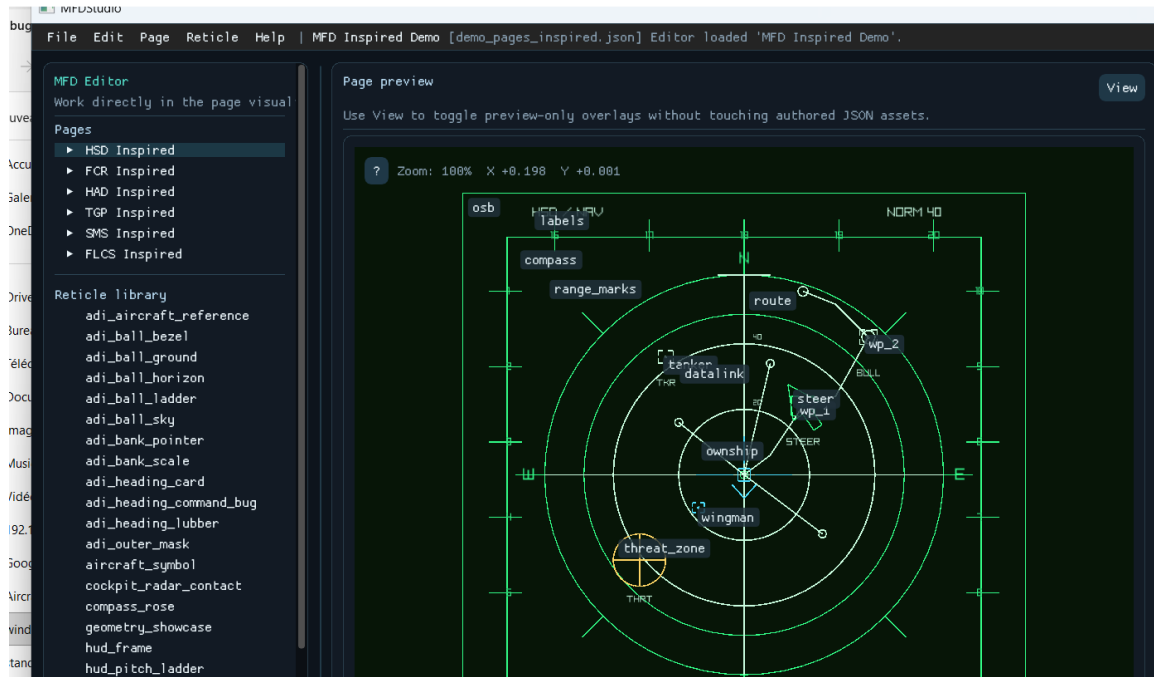
```
cmake --build --preset debug-win32 --target client_mockup_minimal
```

3.4 Tree structure to know

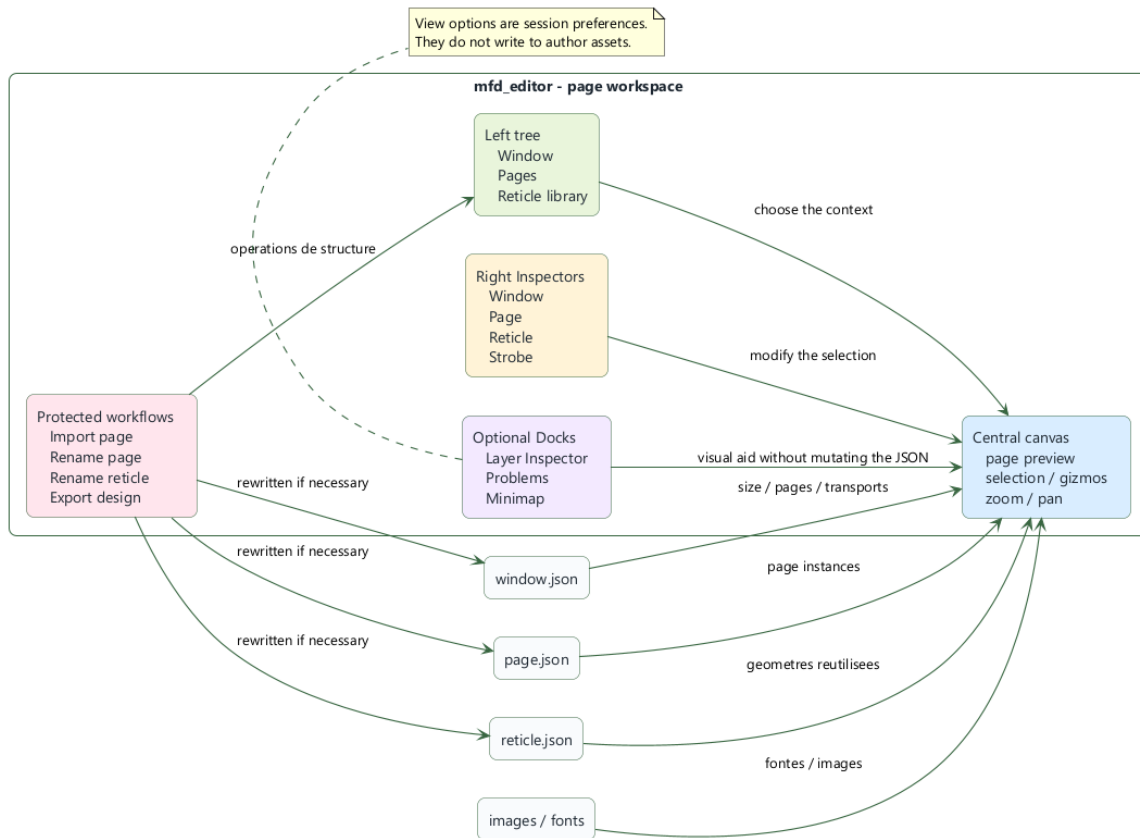
Case	Expected content
assets/windows	JSON root windows
assets/pages	JSON pages
assets/reticles	reticle templates
assets/fonts	possible fonts
examples	reference clients and plugins
Scripts	launch batches
_Exec	runtime staging regenerated by the build

Recommendation: Author in assets/and not in _Exec . _Exec is a runtime staging area, not the long-term source of truth.

4. Use mfd_editor to design a window and its pages



``mfd_editor`` screenshot



Editor workspace

This section describes how to use `mfd_editor` in detail to author a window, its pages, its reticles, and the associated asset-maintenance workflows.

4.1 Launch the editor

Once mfd_editor is built:

- launch mfd_editor.exe
- or launch the staged executable under _Exec

At startup, the editor does not automatically reload a copy from _Exec. It opens an empty document until the user:

- load an existing window
- or create a new window

4.2 Anatomy of the interface

The editor is organized around four areas:

Area	Use
left shaft	window, pages, reticle library
central canvas	page preview, selection, gizmos, zoom/pan
straight inspectors	contextual window editing, page, reticle, strobe
optional docks	Layer Inspector, Problems, Minimap

The preview remains the main surface. The Dockes panels reserve a real space and must not hide the canvas.

4.3 Create a new window

Menu:

- File > New window from scratch

The popup allows you to enter:

Field	Sense
Window file	path of the JSON window to create
Window title	title visible from window
Size (px)	window width/height
Position (px)	initial screen position
Font file (optional)	font used for text rendering
Reticle library folder	reusable templates folder

These fields correspond directly to the root of the JSON window.

4.4 Configure UDP transports

The editor exposes two configuration groups.

Incoming commands to window

Option	Effect
Enable command UDP	activates the command entry point
Command address	IPv4 listening or link address
Command port	UDP port
Command max packet	max size of a datagram

Feedback coming out of runtime to clients

Option	Effect
Enable feedback UDP	activates the return flow
Feedback address	target IPv4 address
Feedback port	UDP feedback port
Feedback max packet	max datagram size

For a single position, use 127.0.0.1.

4.5 Create a first page

During window creation, you can activate Create one initial page then enter:

Field	Sense
First page name	public page name
First page title	human title
First page file	JSON page path
First page background	background color

This first page becomes the defaultPage.

4.6 Save what the editor writes

The document first exists in memory. Nothing persists until the user does:

- File > Save

The writing may concern:

- the JSON window
- the JSON of the initial page
- reticle templates if the workflow has created them

4.7 Add, choose and organize pages

To add a page:

- Page > New page

To choose the default page:

- select the page in the tree
- enable Default page for this window in the page inspector

4.8 Design of a page: layers, static reticles, dynamic bindings, strobe

Page layers

Each page declares layers orders. They are the runtime source of truth for the drawing order.

Static reticles

A page can contain staticReticles :

- either library template instances
- either reticles inline

Each instance aims at layerId.

Dynamic reticle bindings

Section Dynamic reticles in the page inspector:

1. choose one Reticle template
2. choose one Layer
3. to click Add

Each entry creates a runtime binding, not a static instance drawn by default. The live client will then create the real runtime instances.

Page strobe

A page can expose a strobe optional. The editor allows you to assign a strobe template, its initial position and its capture/magnetization configuration.

4.9 Important page fields

The JSON page supports in particular:

Canonical field	Use
name	public page name
title	human title
backgroundColor	bottom
layers	ordered runtime layers
dynamicReticleBindings	dynamic templates allowed on the page

blinkTypes	page blink types
defaultBlink	blink by default
view.center	center of view
view.zoom	zoom
staticReticles	static reticles
strobe	optional strobe

4.10 Supported primitives for constructing reticles

The families of primitives supported by the author model are:

Primitive	Typical usage	Specific fields
text	labels, values	text, fontSize, letterSpacing
time	clock	format, utc, fontSize, letterSpacing
line	features	start, end
circle	markers	radius
ring	crowns	innerRadius, outerRadius, segments
rectangle	panels	width, height, size
ellipse	oval markers	width, height, radiusX, radiusY
square	square markers	size, width, height
diamond	waypoints	size, width, height
triangle	pointers	points
polyline	roads, frames	points, closed
bezier	curves	controlPoints, segments
arc	sectors, arcs	radius, startAngleDegrees, endAngleDegrees, segments
image	bitmaps	file, width, height, size

Common primitive fields cover:

- position
- rotationDegrees
- scale

- visible
- stroke
- thickness
- lineStyle
- filled
- fill

4.11 Selection and direct editing on the canvas

The page preview supports:

- Ctrl+click to add or remove a reticle from the selection
- Esc to empty the selection
- dragging a selection to move a group
- Ctrl+C, Ctrl+X, Ctrl+V, Del

In case of superimposed reticles:

1. right click in the area
2. choice of the desired reticle in the context menu
3. copy/cut/delete operations on the current selection

4.12 Menu View from the preview

The available toggles are:

Option View	Effect
Layer Inspector	layer dock and focus by layer
Minimap	mini navigation view
Problems	dockes diagnostics
Highlight reticle usages	highlights the pages that reference the selected template
Reticle names	displays names
Gizmos	shows visual manipulations
Page context	preview context linked to the page

These options are session preferences. They do not rewrite the author JSON.

Fullscreen preview

The button [], F11 Or Esc allows you to:

- hide sidebar, inspectors and docks
- keep the canvas interactive
- restore the previous state then

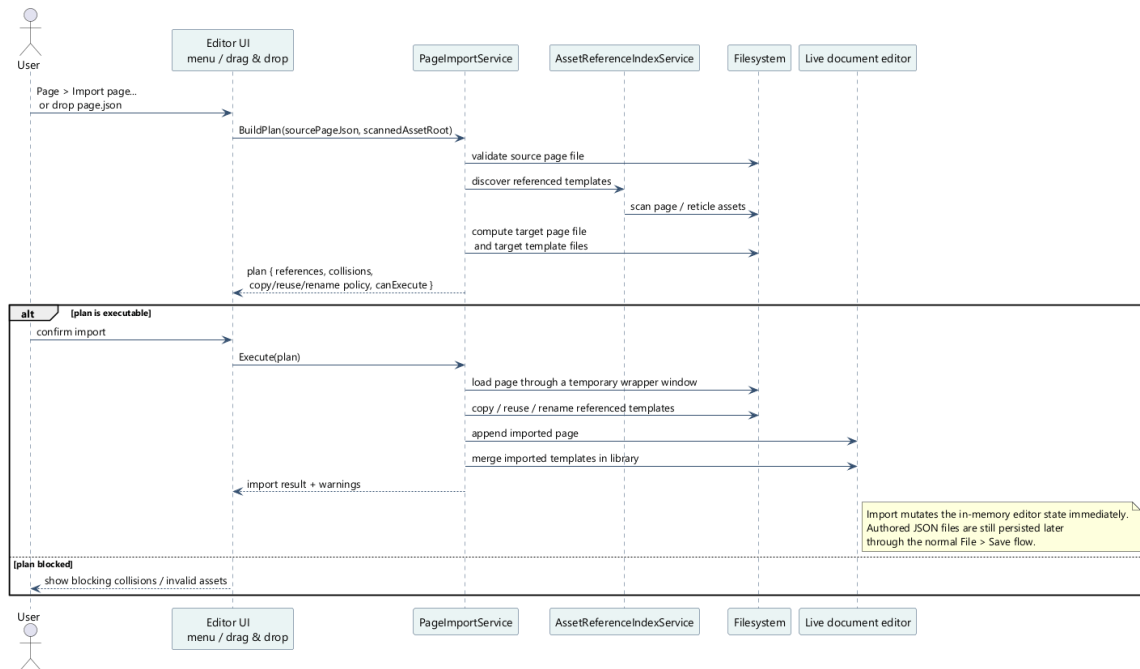
4.13 Layer Focus Mode

THE Layer Inspector allow :

- duty Full View
- to list the runtime layers
- to display a thumbnail and a counter per layer
- to focus a layer to leave only its reticles editable

The other visible layers remain rendered, but dimmed.

4.14 Import an existing page



Page import sequence in the editor

Entries:

- Page > Import page...
- or drag and drop a page JSON

The workflow calculates a plan before execution:

- source page
- target in current tree
- template references
- collisions
- copy/reuse/rename strategy

The current rules are:

- target file missing: copy
- identical file already present: reuse
- different file already present: creation of a renamed copy

4.15 Delete or detach a page

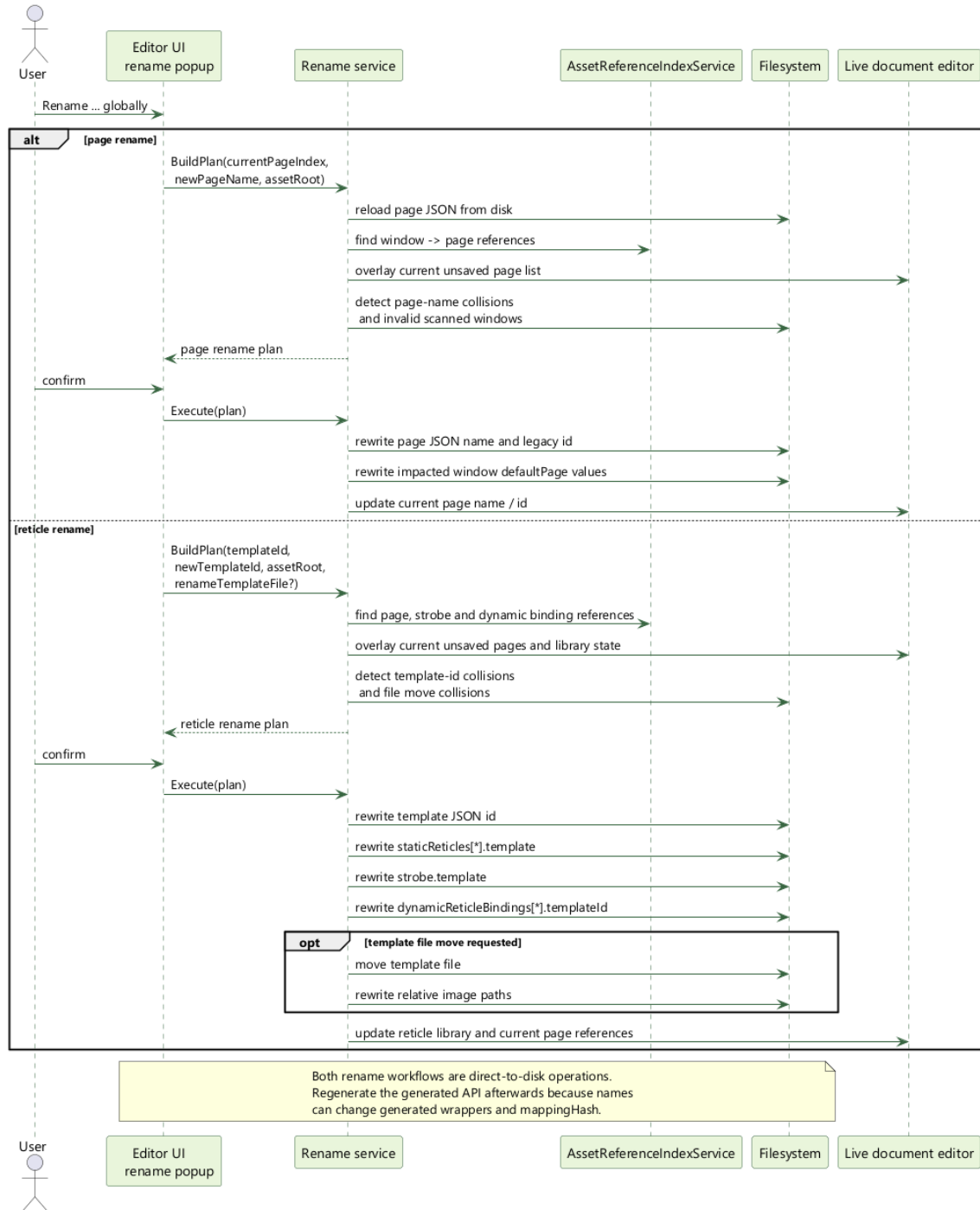
Two distinct actions exist:

Action	Effect
Remove page from window	removes the page from the current window but keeps the file
Delete page asset...	removes the page and marks its JSON for deletion

Security guards:

- the window must keep at least one page
- we do not delete the default page without choosing another one
- deletion out assets/ blocked without explicit override

4.16 Properly rename a shared page



Global page/reticle rename sequence

Entries:

- right click page Rename page globally...
- inspector button
- menu Page > Rename current page globally...

The rename service:

- rescan references window -> page

- rewrites the page JSON
- rewrite the defaultPage impacts
- blocks collisions before any mutation
- warns that the generated API and the mappingHash can change

4.17 Properly renaming a shared reticle template

Entries:

- right click template Rename reticle globally...
- inspector button
- menu Reticle > Rename selected library reticle globally...

The workflow:

- rewrite the id of the template
- rewritten staticReticles[*].template
- rewritten strobe.template
- rewritten dynamicReticleBindings[*].templateId
- can also rename the template file
- updates relative image paths if file moves

4.18 Highlight use of a template

Normal flow:

1. select a reticle template in the tree
2. enable View > Highlight reticle usages
3. the editor highlights:

- the pages that reference this template
- the corresponding instances on the active page

4.19 Export of design documentation

Menu:

- File > Export > Export design...

The exported package may contain:

Exit	Content
README.md	package entry exports
window_icd.md	window view
pages/*.md	detailed views per page
images/*_design_exploded.png	exploded views if rendering was successful
data/design_manifest.json	design manifesto exports

Options available in the popup:

- Markdown ICD snippets
- exploded views
- contact details
- strobe
- blink
- primitive ids
- hash mapping

4.20 Recommended authoring checklist

17. 1. create the window and its transports
18. 2. define the pages and defaultPage
19. 3. declare runtime layers
20. 4. compose static reticles and only expose useful primitives to the client
21. 5. declare the dynamicReticleBindings necessary
22. 6. set strobe if page needs it
23. 7. check Problems, Layer Inspector, Minimap
24. 8. save as assets/
25. 9. regenerate the API if the exposed runtime surface has changed

5. Generate the client API from the generator

This section covers the official generator `mfd_client_api/generator`.

5.1 Entry, exit, contract

The generator takes as input:

- a root window JSON

It produces:

- a generated C++ header
- a generated C++ source
- A `.generated.map` optional but highly recommended

The generator parses:

- the window
- the referenced pages
- the reticle library

He deduces:

- the pages exposed
- static reticles
- the primitives exposed
- dynamic templates
- the blink types
- stable transport IDs
- THE mappingHash

5.2 Official CMake macro

The normal path is the `client_api_generate_ui(...)` CMake macro.

Complete example:

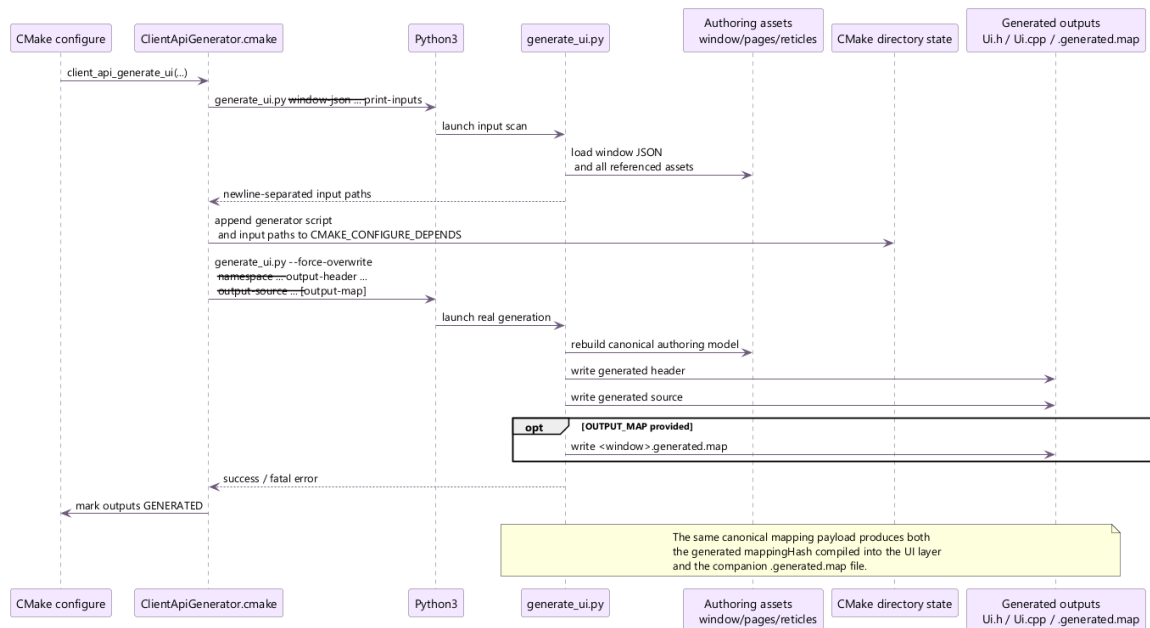
```
client_api_generate_ui(
    WINDOW_JSON "assets/windows/demo_pages_cockpit.json"
    OUTPUT_HEADER "${CMAKE_CURRENT_SOURCE_DIR}/generated/MockupUi.h"
    OUTPUT_SOURCE "${CMAKE_CURRENT_SOURCE_DIR}/generated/MockupUi.cpp"
    OUTPUT_MAP "${MFD_ROOT_DIR}/assets/windows/demo_pages_cockpit.generated.map"
    NAMESPACE "mockup_ui"
    UI_CLASS_NAME "CockpitMockupUi"
    HEADER_INCLUDE "MockupUi.h")
```

5.3 client_api_generate_ui arguments

Argument	Mandatory	Use
WINDOW_JSON	yes	root window to parse
OUTPUT_HEADER	yes	generated header

OUTPUT_SOURCE	yes	generated source
OUTPUT_MAP	recommended	generated transport map
NAMESPACE	no	generated C++ namespace
UI_CLASS_NAME	no	explicit UI root class name
PAGE_CLASS_SUFFIX	no	generated page class suffix
UI_CLASS_SUFFIX	no	generated UI root suffix when UI_CLASS_NAME is omitted
HEADER_INCLUDE	no	header name included by the generated .cpp
CONFIGURE_DEPENDS	no	extra CMake regeneration dependencies

5.4 What the CMake macro does



Client API generation sequence

The macro:

26. 1. resolves the generator script path
27. 2. resolves source and output paths
28. 3. locates a Python 3 interpreter
29. 4. runs an input scan via --print-inputs
30. 5. records those inputs in CMAKE_CONFIGURE_DEPENDS
31. 6. runs the real generation with --force-overwrite

Result:

- when one source asset changes, CMake knows it must regenerate
- generated outputs stay marked as GENERATED

5.5 Underlying Python script

The main script is:

```
mfd_client_api/generator/scripts/generate_ui.py
```

Main CLI interface:

```
--window-json
--output-header
--output-source
--output-map
--namespace
--ui-class-name
--page-class-suffix
--ui-class-suffix
--header-include
--print-inputs
--force-overwrite
```

5.6 Important generation rules

The generator:

- derives transport IDs from stable hashes of canonical keys
- emits the mappingHash from the canonical mapping payload
- rejects ID collisions
- checks the uniqueness of generated C++ names
- validates dynamicReticleBindings consistency
- exposes primitives according to the authored asset rules

5.7 Rules for exposing primitives

A primitive becomes a generated client handle when:

- it is explicitly marked exposed
- or the generator deduces that it must be according to the rules in force

The type of handle issued depends on the type of primitive:

Primitive author	Handle generated
text	TextHandle
time	TimeHandle
line	LineHandle
circle	CircleHandle
ring	RingHandle

rectangle	RectangleHandle
ellipse	EllipseHandle
square	SquareHandle
diamond	DiamondHandle
triangle	TriangleHandle
polyline	PolylineHandle
bezier	BezierHandle
arc	ArcHandle
image	ImageHandle

5.8 What the generated API contains

The generated API typically exposes:

- a UI root
- one class per page
- one class per static reticle
- a dynamic set per dynamic template
- one handle per exposed primitive
- of the BlinkType of page
- A StrobeHandle of page

5.9 What it contains .generated.map

The JSON map contains:

Painting	Content
pages	Stable page IDs
reticles	Stable static reticle IDs
primitives	Stable IDs of exposed primitives
templates	Stable IDs of dynamic templates
blinkTypes	Stable IDs of blink types

The top-level also contains:

- schemaVersion
- mappingHash
- window metadata

5.10 Example of integration into a CMake target

Minimal extract:

```
set(MY_GENERATED_DIR ${CMAKE_CURRENT_SOURCE_DIR}/generated)

client_api_generate_ui(
    WINDOW_JSON "assets/windows/my_window.json"
    OUTPUT_HEADER "${MY_GENERATED_DIR}/MyWindowUi.h"
    OUTPUT_SOURCE "${MY_GENERATED_DIR}/MyWindowUi.cpp"
    OUTPUT_MAP "${MFD_ROOT_DIR}/assets/windows/my_window.generated.map"
    NAMESPACE "my_window_ui"
    UI_CLASS_NAME "MyWindowUi"
    HEADER_INCLUDE "MyWindowUi.h")

add_executable(my_client
    src/main.cpp
    ${MY_GENERATED_DIR}/MyWindowUi.cpp
    ${MY_GENERATED_DIR}/MyWindowUi.h)

target_include_directories(my_client PRIVATE ${MY_GENERATED_DIR})
target_link_libraries(my_client PRIVATE mfd::client mfd::io_json)
```

5.11 When to regenerate

You must regenerate if you modify:

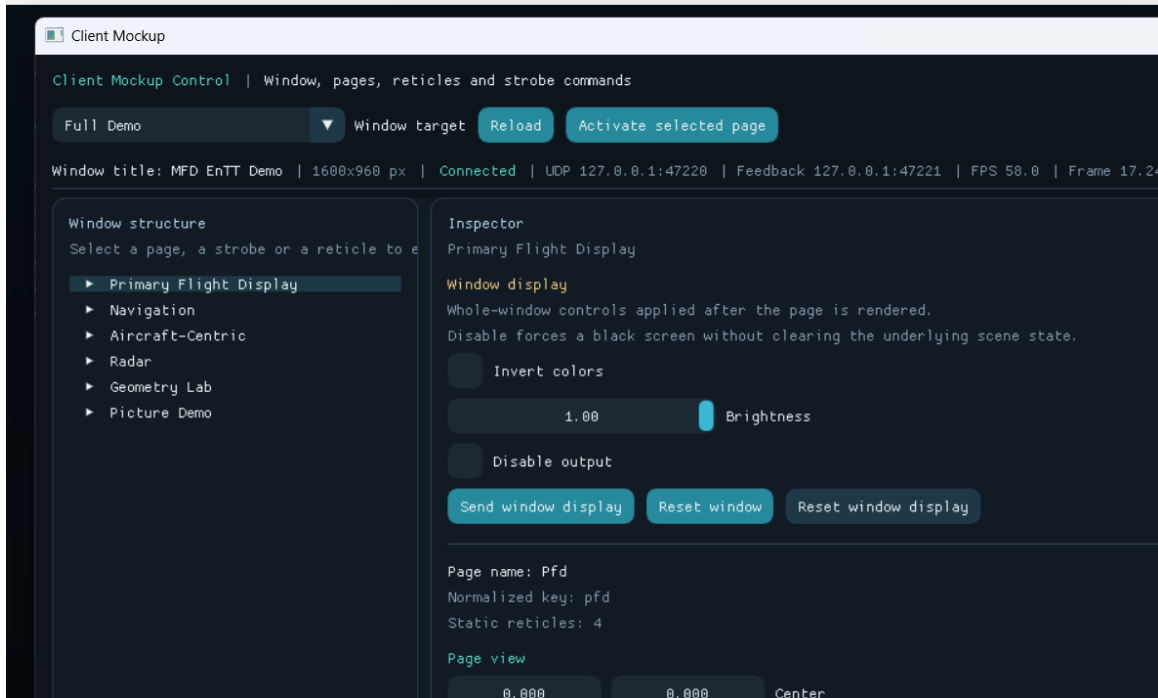
- the name of a page
- the name of a reticle template
- the primitives exposed
- the blink types
- dynamic bindings
- the visible structure of the window

5.12 Common causes of generation failure

Cause	Effect
JSON invalid window or page	parse failure
C++ generated name collision	generator abort
dynamicReticleBindings inconsistent	generator abort
unknown template	abortion
already existing output file without overwrite	abortion

Important: The generator is not a cosmetic convenience. It formalizes the customer contract. Any author development that changes this contract must be considered as an interface change.

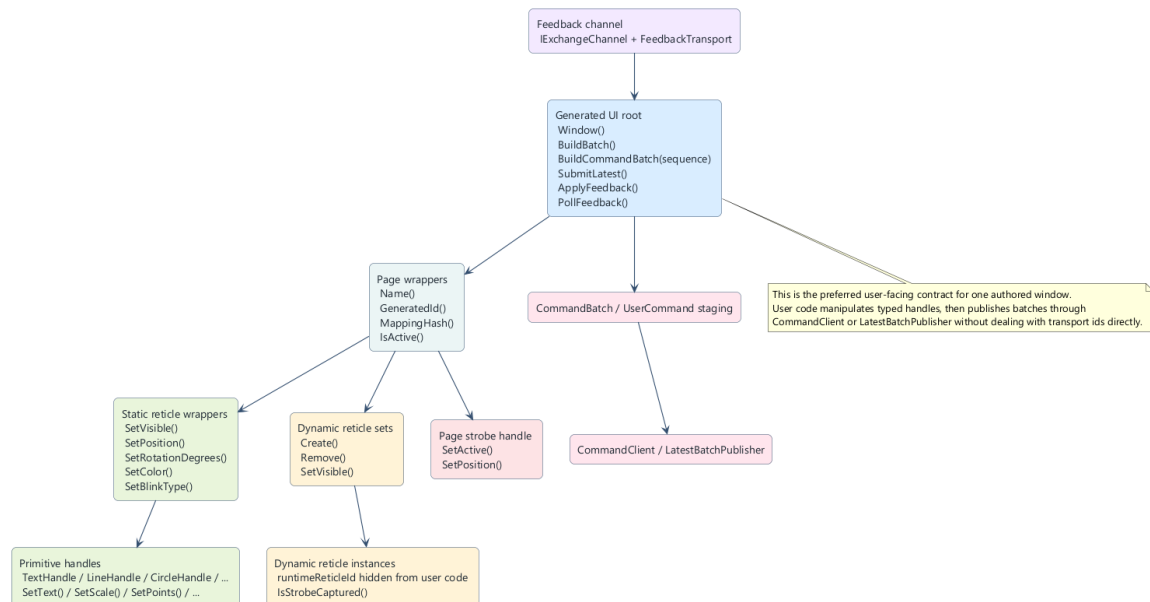
6. Use the generated API to communicate with a window



``client_mockup` runtime client screenshot`

This section details the recommended way to drive a runtime window via the generated API.

6.1 Two layers to distinguish



Generated API user contract

You should distinguish two layers:

- the generated layer, which is oriented around authored concepts
- CommandClient, which remains the final transport-sending boundary

The recommended usage is:

32. 1. mutate the generated handles
33. 2. build a batch
34. 3. send it through CommandClient or LatestBatchPublisher

6.2 Load window configuration

The loading entry point is `mfd::JsonLoader`.

```
#include "mfd/io/JsonLoader.h"

mfd::JsonLoader loader;
const auto loaded = loader.LoadWindowConfiguration("assets/windows/demo_pages_cockpit.json");
```

The result contains:

- `loaded.window`
- `loaded.document`
- `loaded.generatedTransportMap`

6.3 Create the CommandClient

```
#include "mfd/control/CommandClient.h"

if (!loaded.window.commandTransports.udp.has_value() || !loaded.generatedTransportMap.has_value())
{
    return 1;
}

mfd::CommandClient client(*loaded.window.commandTransports.udp, loaded.generatedTransportMap);
if (!client.IsReady())
{
    // Inspect client.LastError()
    return 1;
}
```

6.4 Generated UI root

Example with a generated UI:

```
#include "MockupUi.h"

mockup_ui::CockpitMockupUi ui;
```

Recommended startup skeleton:

```
auto& radar = ui.Radar();

radar.fixedTrackAlpha.SetVisible(true);
radar.fixedTrackAlpha.TrackLabel().SetText("MOCK");

client.ActivatePage(radar);
client.SetPageView(radar, {0.0f, 0.0f}, 1.0f);
client.SendBatch(ui.BuildCommandBatch(42U));
```

The UI root generally exposes:

Method	Role
Window()	patch whole-window display
page accessors	typed navigation
BuildBatch()	builds the staged UserCommand list
BuildCommandBatch(sequence)	builds a CommandBatch carrying the generated mappingHash
SubmitLatest(publisher, sequence)	publishes through LatestBatchPublisher
ApplyFeedback(...)	integrates decoded feedback
ApplyFeedbackPayload(...)	decodes one raw payload
PollFeedback(...)	drains a feedback channel
Reset()	resets the local staged state

6.5 Page wrappers

Each generated page typically exposes:

Member / method	Role
Name()	authored page name
GeneratedId()	stable transport ID
MappingHash()	compatibility hash
IsActive()	authoritative runtime state when feedback is connected
static reticle members	direct access
strobe	StrobeHandle of page
dynamic sets	one per dynamic template
blink types	BlinkType of page

6.6 Activate a page and adjust its view

The recommended route is to pass the generated page wrapper:

```
auto& radar = ui.Radar();
client.ActivatePage(radar);
client.SetPageView(radar, {0.0f, 0.0f}, 1.0f);
```

This forces the use:

- of the generated page ID
- of mappingHash partner

6.7 Modify the overall window display

The UI root exposes Window():

```
ui.Window().SetColorInverted(false);
ui.Window().SetBrightness(0.65f);
ui.Window().SetDisabled(false);
client.SendBatch(ui.BuildCommandBatch(42U));
```

6.8 Mutate a static reticle

Example :

```
auto& radar = ui.Radar();

radar.fixedTrackAlpha.SetVisible(true);
radar.fixedTrackAlpha.SetBlinkType(radar.fast);
radar.fixedTrackAlpha.SetPosition({0.30f, 0.18f});
radar.fixedTrackAlpha.SetRotationDegrees(-15.0f);
radar.fixedTrackAlpha.SetColor({77, 224, 255, 255});
radar.fixedTrackAlpha.TrackLabel().SetText("MOCK");

client.SendBatch(ui.BuildBatch());
```

Static wrappers derive from the mfd::client::Reticle contract.

Common reticle surface

Method	Effect
SetVisible	visibility
SetBlinkEnabled	activate or not the blink
SetBlink	active + type
SetBlinkType	explicit blink type
ClearBlinkType	return to default blink page
SetPosition	logical position
SetRotationDegrees	rotation
SetColor	color stroke
SetThickness	thickness
SetText	text
SetLetterSpacing	text spacing

6.9 Using exposed primitive handles

The great advantage of the generated API is to drive an exposed primitive without rebuilding the entire reticle.

Common primitive surface

Method	Effect
SetVisible	primitive visibility
SetPosition	local translation
SetRotationDegrees	local rotation
SetScale	ladder
SetColor	line color
SetFillColor	fill color
SetFilled	fill flag
SetThickness	thickness
SetLineStyle	solid, dotted, dashed

Specialized surfaces

Handle	Extensions
TextHandle	SetText, SetLetterSpacing
TimeHandle	SetLetterSpacing
LineHandle	SetStart, SetEnd
CircleHandle	SetRadius
RingHandle	SetInnerRadius, SetOuterRadius, SetSegments
RectangleHandle	SetWidth, SetHeight, SetSize
EllipseHandle	SetWidth, SetHeight, SetSize
SquareHandle	SetWidth, SetHeight, SetSize
DiamondHandle	SetWidth, SetHeight, SetSize
TriangleHandle	SetPoints
PolylineHandle	SetPoints, SetClosed

BezierHandle	SetControlPoints, SetSegments
ArcHandle	SetRadius, SetStartAngleDegrees, SetEndAngleDegrees, SetSegments
ImageHandle	common area only

Example of exposed primitive

```
auto& picture = ui.PictureDemo().pictureDemo;
picture.DemoPicture().SetVisible(true);
picture.DemoPicture().SetPosition({0.0f, 0.0f});
picture.DemoPicture().SetScale({1.10f, 1.10f});
picture.DemoPicture().SetRotationDegrees(8.0f);
client.SendBatch(ui.BuildBatch());
```

6.10 Manage dynamic reticles

The recommended route is the generated dynamic set.

```
auto& tracks = ui.Radar().DynamicRadarTrack();
auto& track = tracks.Create();

track.SetVisible(true);
track.SetPosition({0.18f, -0.24f});
track.SetRotationDegrees(55.0f);
track.TrackLabel().SetText("B21");

client.SendBatch(ui.BuildBatch());
```

To delete one instance:

```
tracks.Remove(track);
client.SendBatch(ui.BuildBatch());
```

Surface of one generated dynamic set

Method	Role
Create()	allocates a typed handle and a runtime id cache
Remove(handle)	deletes one instance
SetVisible(bool)	hides or shows the whole dynamic family
AppendCommands	builds runtime commands

What the dynamic set hides

The business code does not have to manage:

- the runtimeReticId
- the mappingHash
- transport template ID

6.11 Publish in batches

Two forms are common.

Batch without sequence

```
client.SendBatch(ui.BuildBatch());
```

Batch with sequence and mappingHash

```
const auto batch = ui.BuildCommandBatch(42U);
client.SendBatch(batch);
```

The sequence is useful to:

- identify one external cycle
- group one coherent burst of mutations

6.12 Use LatestBatchPublisher

When a real-time client continuously computes the latest state, LatestBatchPublisher lets you publish the freshest batch through one dedicated path.

```
#include "mfd/client/LatestBatchPublisher.h"

mfd::client::LatestBatchPublisher publisher(*loaded.window.commandTransports.udp);
if (!publisher.IsReady())
{
    return 1;
}

ui.SubmitLatest(publisher, 43U);
```

6.13 Use strobe and feedback

The strobe remains page-scoped.

```
auto& radar = ui.Radar();
if (radar.strobe.IsValid())
{
    radar.strobe.SetActive(true);
    radar.strobe.SetPosition({0.15f, -0.08f});
    client.SendBatch(ui.BuildBatch());
}
```

Runtime feedback can then be connected:

```
#include "mfd/control/FeedbackTransport.h"

auto feedbackChannel = mfd::CreateFeedbackReceiverChannel(*loaded.window.feedbackTransports.udp);
std::string error;
ui.PollFeedback(*feedbackChannel, 8U, &error);

const bool pageActive = ui.Radar().IsActive();
```

For a dynamic reticle:

```
const bool captured = track.IsStrobeCaptured();
```

6.14 What feedback guarantees

Page::IsActive() is true only when the runtime confirms that the page is currently rendered as active.

DynamicReticle::IsStrobeCaptured() is true only when the latest strobe feedback points to that exact runtime instance.

6.15 Client sanitization and hygiene

The reference client applies safeguards that are worth keeping:

- clamp positions in [-1, 1]
- clamp brightness to [0, 1]
- zoom clean before sending
- rejection of empty dynamic ids
- verification of client.IsReady()
- display of client.LastError() on failure

6.16 When to use raw API

The Raw API CommandClient by author names remains useful for:

- generic tools
- transition code
- low level debugging

Example :

```
mfd::ReticlePatch patch;
patch.visible = true;
patch.position = mfd::Vec2 {0.25f, 0.10f};
patch.text = std::string {"T42"};

client.UpsertDynamicReticle("Radar", "track_42", "radar_track", patch);
```

But for a window-specific client, the generated path remains the right one.

6.17 Minimal end-to-end example

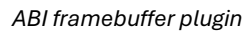
```
#include "MockupUi.h"
#include "mfd/control/CommandClient.h"
#include "mfd/io/JsonLoader.h"

int main()
{
    mfd::JsonLoader loader;
    const auto loaded = loader.LoadWindowConfiguration("assets/windows/demo_pages_cockpit.json");
    if (!loaded.window.commandTransports.udp.has_value() || !loaded.generatedTransportMap.has_value())
    {
        return 1;
    }

    mfd::CommandClient client(*loaded.window.commandTransports.udp, loaded.generatedTransportMap);
    if (!client.IsReady())
```

```
{  
    return 1;  
}  
  
mockup_ui::CockpitMockupUi ui;  
auto& cockpit = ui.Cockpit();  
  
client.ActivatePage(cockpit);  
ui.Window().SetBrightness(0.65f);  
cockpit.strobe.SetActive(true);  
cockpit.strobe.SetPosition({0.12f, -0.05f});  
  
client.SendBatch(ui.BuildCommandBatch(1U));  
return 0;  
}
```

Important: The recommended model is not "directly call the network everywhere". The recommended model is "mutate a local generated state, then build a coherent batch".



7.1 Why a stable C ABI

- STL incompatibilities
- C++ ABI differences
- exceptions across the DLL boundary
- sharing host-owned graphic objects

```
mfd_window_plugin_api/include/mfd/window/WindowLauncherPlugin.h
```

7.2 Mandatory entry point

MfdGetWindowFramebufferPluginApi

MFD_WINDOW_FRAMEBUFFER_PLUGIN_ENTRY_POINT

7.3 Structure MfdWindowFramebufferPluginApi

Field	Role
struct_size	structure versioning
info	plugin metadata
plugin_context	plugin-owned opaque context

init	plugin initialization
submit_frame	per-frame callback
close	shutdown callback
destroy	final context destruction

7.4 Structure MfdWindowFramebufferPluginInfo

Field	Role
struct_size	versioning
abi_version	expected ABI
plugin_id	stable identifier
display_name	human-readable display name

7.5 Structure MfdWindowFramebufferFrame

The runtime returns a raw frame descriptor:

Field	Meaning
struct_size	structure size
pixel_format	pixel format, currently MfdWindowFramebufferPixelFormat_Rgba32
width	width in pixels
height	height in pixels
row_stride_bytes	stride per line
pixels	raw pointer to bytes
pixel_bytes	total number of bytes available

7.6 Validation helpers provided

The header also exports C helpers:

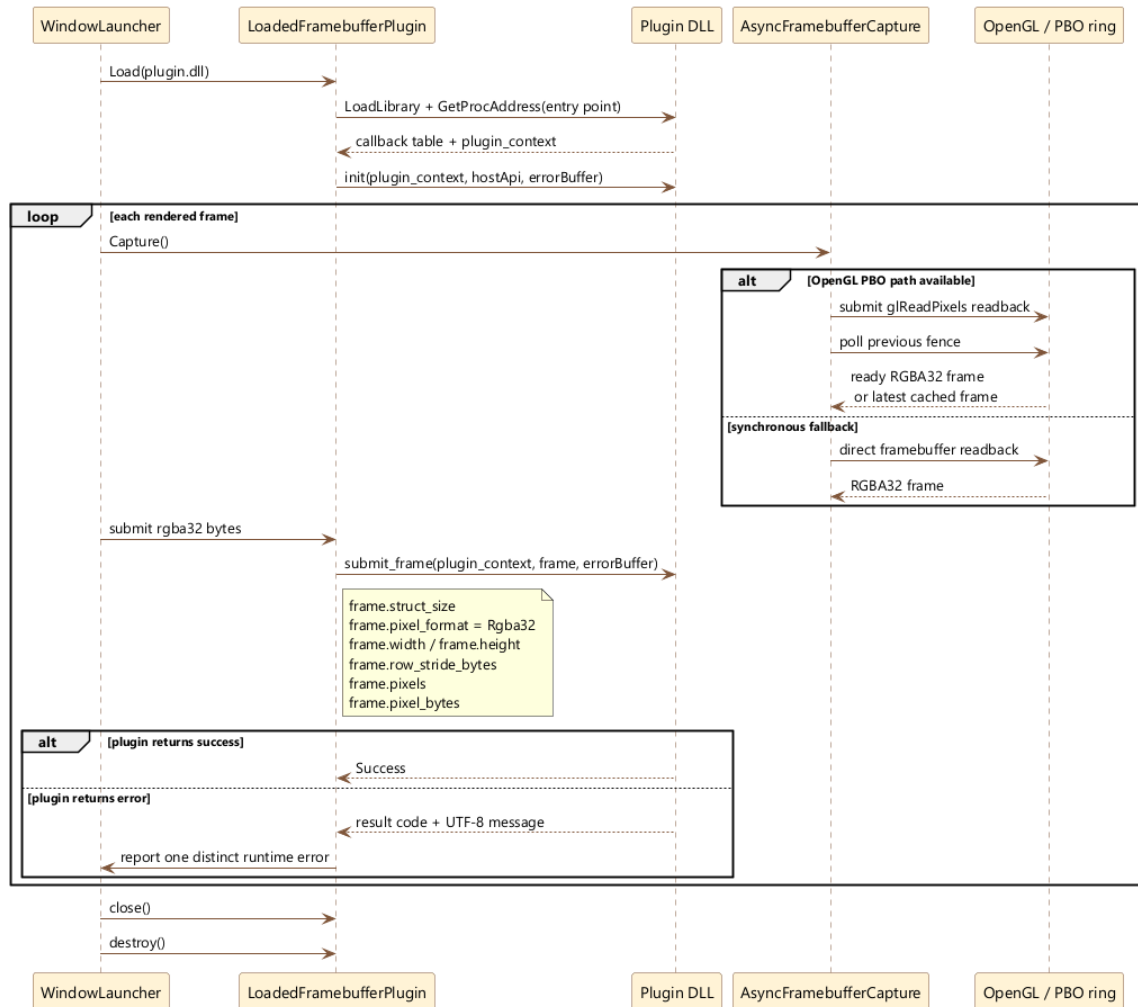
Function	Use
MfdWindowComputeFramebufferRgba32ByteCount	calculation of expected size
MfdWindowValidateFramebufferRgba32Layout	checks a raw RGBA32 buffer

MfdWindowValidateFramebufferFrame

validate one frame structure

They must be used defensively in the plugin.

7.7 Plugin lifecycle



Framebuffer capture runtime sequence and plugin

The normal lifecycle is:

35. 1. mfd_window loads the DLL
36. 2. the runtime calls MfdGetWindowFramebufferPluginApi
37. 3. the plugin allocates its context
38. 4. the runtime calls init
39. 5. the runtime calls submit_frame every frame
40. 6. the runtime calls close at shutdown
41. 7. the runtime calls destroy to free the context

7.8 Minimal plugin skeleton

```
#include "mfd/window/WindowLauncherPlugin.h"

struct MyPluginContext
```

```

{
    bool initialized = false;
};

MfdWindowFramebufferPluginResultCode MFD_WINDOW_PLUGIN_CALL InitPlugin(
    void* pluginContext,
    const MfdWindowFramebufferPluginHostApi* host,
    MfdWindowUtf8Buffer* error) noexcept
{
    if (pluginContext == nullptr || host == nullptr ||
        host->abi_version != MFD_WINDOW_FRAMEBUFFER_PLUGIN_ABI_VERSION)
    {
        return MfdWindowFramebufferPluginResultCode_InvalidArgument;
    }

    static_cast<MyPluginContext*>(pluginContext)->initialized = true;
    return MfdWindowFramebufferPluginResultCode_Success;
}

MfdWindowFramebufferPluginResultCode MFD_WINDOW_PLUGIN_CALL SubmitFramePlugin(
    void* pluginContext,
    const MfdWindowFramebufferFrame* frame,
    MfdWindowUtf8Buffer* error) noexcept
{
    if (pluginContext == nullptr || frame == nullptr)
    {
        return MfdWindowFramebufferPluginResultCode_InvalidArgument;
    }

    if (MfdWindowValidateFramebufferFrame(frame) == 0)
    {
        return MfdWindowFramebufferPluginResultCode_InvalidArgument;
    }

    // Copier ou consommer frame->pixels ici.
    return MfdWindowFramebufferPluginResultCode_Success;
}

void MFD_WINDOW_PLUGIN_CALL ClosePlugin(void* pluginContext) noexcept
{
    (void)pluginContext;
}

void MFD_WINDOW_PLUGIN_CALL DestroyPlugin(void* pluginContext) noexcept
{
    delete static_cast<MyPluginContext*>(pluginContext);
}

extern "C" __declspec(dllexport) MfdWindowFramebufferPluginResultCode MFD_WINDOW_PLUGIN_CALL
MfdGetWindowFramebufferPluginApi(MfdWindowFramebufferPluginApi* outApi, MfdWindowUtf8Buffer* error)
noexcept
{
    if (outApi == nullptr)
    {
        return MfdWindowFramebufferPluginResultCode_InvalidArgument;
    }

    auto* context = new MyPluginContext();
    *outApi = {};
}

```

```

outApi->struct_size = sizeof(*outApi);
outApi->info.struct_size = sizeof(outApi->info);
outApi->info.abi_version = MFD_WINDOW_FRAMEBUFFER_PLUGIN_ABI_VERSION;
outApi->plugin_context = context;
outApi->init = &InitPlugin;
outApi->submit_frame = &SubmitFramePlugin;
outApi->close = &ClosePlugin;
outApi->destroy = &DestroyPlugin;
return MfdWindowFramebufferPluginResultCode_Success;
}

```

7.9 Example of repository reference plugin

The repository provides:

```
examples/mfd_framebuffer_stdout_plugin/src/FramebufferStdoutPlugin.cpp
```

This plugin:

- validate the frame
- displays a single line in the first frame
- illustrates context allocation and destruction

7.10 What to do in submit_frame

In submit_frame, you should:

42. 1. check pluginContext != nullptr
43. 2. check frame != nullptr
44. 3. call MfdWindowValidateFramebufferFrame(frame)
45. 4. consume or copy frame->pixels
46. 5. return a clear error code on failure

7.11 What not to do

You should not:

- keep a pointer frame->pixels after return of callback without copy
- throw exceptions through the ABI
- assume another format than RGBA32
- assume the host is sharing GPU objects
- write to the memory pointed to by pixels

7.12 Practical consumption strategy

Depending on the need, the plugin can:

- copy the frame to a private CPU buffer
- push the frame to an internal lock-free queue
- convert the frame for an encoder
- publish the frame to another IPC system

The important rule is: the lifetime of raw bytes is only guaranteed during the call.

7.13 CMake build of the plugin

The reference plugin is a classic C++17 DLL. The critical point is not CMake itself, but:

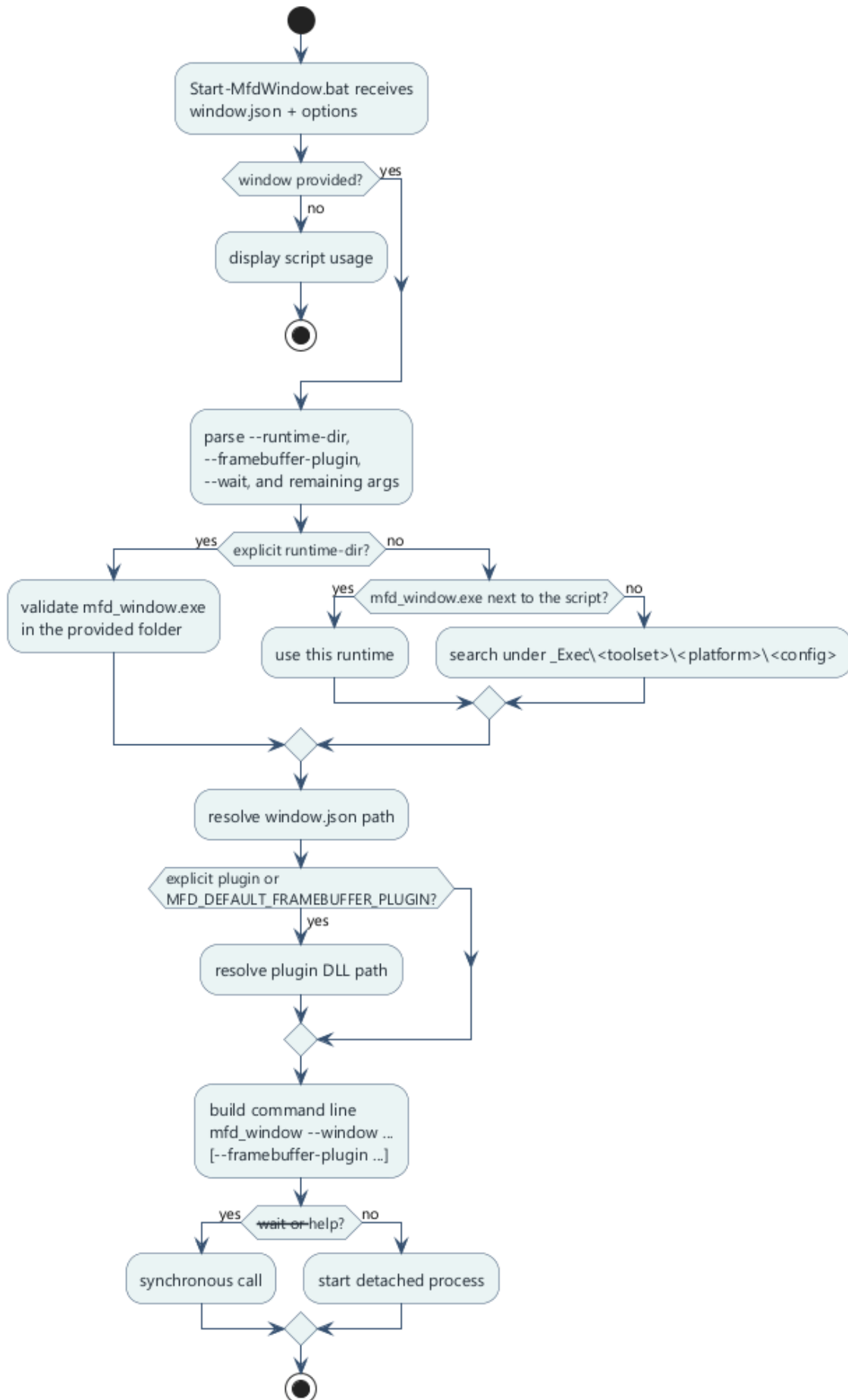
- exporting the symbol MfdGetWindowFramebufferPluginApi
- the inclusion of the public header of the API plugin
- strict compliance with the ABI

7.14 Recommended testing strategy

To validate a framebuffer plugin:

47. 1. start with a "stdout" or "log" plugin
48. 2. check that init is called
49. 3. check that submit_frame receives the expected dimensions
50. 4. check the byte size via the validation helper
51. 5. then connect the consumption business logic

8. Launch a window with a script and a framebuffer plugin



Launch script flow

The repository provides a reusable batch launcher:

```
Scripts/Start-MfdWindow.bat
```

8.1 Use of the script

The script expects at least:

```
Start-MfdWindow.bat <window.json>
```

Full usage:

```
Start-MfdWindow.bat <window.json> [--runtime-dir <dir>] [--framebuffer-plugin <plugin.dll>] [--wait]
[extra launcher args]
```

8.2 Supported arguments

Argument	Effect
<window.json>	window to launch
--runtime-dir <dir>	force the folder containing mfd_window.exe
--framebuffer-plugin <plugin.dll>	explicitly injects a framebuffer plugin
--wait	synchronous runtime call
extra launcher args	args transmitted to the launcher

8.3 Runtime resolution

The script is looking for mfd_window.exe in this order:

52. 1. the folder passed via --runtime-dir
53. 2. the script folder if mfd_window.exe is present there
54. 3. _Exec/<toolset>/<platform>/<config>
55. 4. a fallback scan under _Exec

8.4 Plugin resolution

The plugin can come:

- from --framebuffer-plugin
- or from the MFD_DEFAULT_FRAMEBUFFER_PLUGIN environment variable

The script then resolves the path:

- absolute if given directly
- relative to the runtime folder
- relating to the root of the deposit
- relative to the script folder
- relating to the current file

8.5 Example with the minimal script from the repository

Scripts/Start-MfdMinimal.bat preconfigured:

```
@echo off
setlocal
set "MFD_DEFAULT_FRAMEBUFFER_PLUGIN=mfd_framebuffer_stdout_plugin.dll"
call "%~dp0Start-MfdWindow.bat" "assets/windows/demo_pages_minimal.json" %*
exit /b %ERRORLEVEL%
```

So the simplest call is:

```
.\Scripts\Start-MfdMinimal.bat
```

This is functionally equivalent to launching mfd_window with:

- the window assets/windows/demo_pages_minimal.json
- the plugin mfd_framebuffer_stdout_plugin.dll

8.6 Explicit example with custom plugin

```
.\Scripts\Start-MfdWindow.bat ` "assets/windows/demo_pages_minimal.json" `
  --framebuffer-plugin "build\vs2022-win32\examples\Debug\my_framebuffer_plugin.dll" `--wait``## 8.7
Command line equivalent` mfd_window `The runtime directly accepts:`` powershell
mfd_window --window assets/windows/demo_pages_minimal.json --framebuffer-plugin
my_framebuffer_plugin.dll
```

The launcher usage text also covers:

- --help
- the absence or not of a default window
- the shortcuts F1, R, 1..9

8.8 Write your own project launch script

A clean project script generally contains:

56. 1. the target window
57. 2. an optional default plugin variable
58. 3. the call has Start-MfdWindow.bat

Example :

```
@echo off
setlocal
set "MFD_DEFAULT_FRAMEBUFFER_PLUGIN=my_framebuffer_plugin.dll"
call "%~dp0Start-MfdWindow.bat" "assets/windows/my_window.json" %*
exit /b %ERRORLEVEL%
```

8.9 Launch checklist with plugin

Before launching, check:

- the JSON window exists
- mfd_window.exe was well constructed
- the plugin DLL exists in a resolvable path
- the symbol MfdGetWindowFramebufferPluginApi is exported

- the ABI of the plugin corresponds to the expected version

8.10 If the plugin does not load

Check in order:

59. 1. correct dll path
60. 2. DLL runtime dependencies present
61. 3. symbol name exports correct
62. 4. struct_size And abi_version correctly informed
63. 5. absence of crashes in init

9. Overall Checklist Recommends

9.1 Complete project workflow

64. 1. author the window and pages in mfd_editor
65. 2. save as assets/
66. 3. expose only the necessary primitives to the client
67. 4. declare dynamic templates and strobe if necessary
68. 5. generate Ui.h, Ui.cpp And <window>.generated.map
69. 6. integrate the generated API into a C++17 client target
70. 7. throw mfd_window
71. 8. post orders via CommandClient
72. 9. plug in UDP feedback if authoritative runtime state is required
73. 10. plug in a framebuffer plugin if the image output needs to be captured

9.2 Design errors to avoid

- author directly in _Exec
- expose too many primitives without business need
- drive text names everywhere while a generated API exists
- keep pointers to the framebuffer after the callback returns
- forget to regenerate the API after renaming the page/template
- mix client business logic and visual authoring

9.3 Most important files to know

File	Why it is important
assets/windows/<window>.json	runtime and generator entry point
assets/windows/<window>.generated.map	stable transport mapping
mfd_client_api/generator/scripts/generate_ui.py	generator source
mfd_common_api/include/mfd/control/CommandClient.h	public transport facade
mfd_client_api/include/mfd/client/Animation.h	public client handles
mfd_window_plugin_api/include/mfd/window/WindowLauncherPlugin.h	ABI framebuffer plugin
Scripts/Start-MfdWindow.bat	resolution and launch logic

9.4 Conclusion

The recommended path in MFDStudio is clear:

- author the structure in the editor
- generate a typical C++ contract from the window
- control the window live in batches
- capture the rendering via a stable ABI plugin if necessary

This flow keeps:

- a modular architecture
- an intentional API surface
- good traceability between assets, client and runtime